

Codex transcript fidelity: from native events to trustworthy queries

1. Mission and scope

A transcript converter is an evidence processor. Its job is to preserve what the agent actually said and executed, and expose those facts in a form that can be queried without reopening a large native log. It must distinguish a command that ran from a command merely mentioned in prose. This ticket repairs gaps exposed while recovering a closed MLX video-embedding side conversation on September 6, 2026.

The installed go-minitrace binary discovered two native Codex files, converted them without identity collisions, and supported SQL over their archives. However, the main file converted to zero conversation turns and 335 tool calls; 327 calls were classified as exec / OTHER. The native source contained real user and assistant messages and structured command-completion events. The problem is incomplete adapter coverage, not a deprecated query engine.

This guide describes a proposed implementation. No adapter repair is delivered by this documentation ticket. Code references were inspected at repository commit `a6acfcc9130f88fd8a6237494c4664d`. The installed CLI identifies itself as dev; its exact build commit was not established. Source inspection independently confirms the relevant gaps in this checkout.

Deliverables for the future implementation are: restored messages, authoritative executed commands and outcomes, accurate turn linkage, multi-file operation evidence, explicit unknown outcomes, conversion diagnostics, and regression tests that exercise both conversion and SQL materialization. Preserve established native session identity and collision safety.

Do not broaden this into recovering the entire MLX conversation or implementing the MLX repair. The inspected main transcript explicitly says someone else is working on that fix. Its absence from the normalized tool list is not proof that all missing side-chat work exists in this file. Discovery of archived or separately stored conversations is a distinct investigation.

2. The system an intern needs to understand

2.1 Native JSONL and redundant representations

JSONL stores one JSON object per line. A line is an event, not necessarily a user-visible turn. Codex may record session metadata, turn context, response items, execution events, usage, and completion notifications. Several events may describe the same logical message or execution.

The inspected file declares Codex CLI 0.153.4 and `history_mode: paginated`. It includes `response_item` messages and `event_msg/item_completed` records with typed items such as `UserMessage`, `AgentMessage`, and `CommandExecution`. Older persisted formats use `event_msg/user_message`, `agent_message`, and `exec_command_end`. File format dispatch currently distinguishes session JSONL, exec JSONL, and legacy rollout inputs; that does not guarantee coverage of every event shape inside session JSONL.

A message index is not a native turn ID. A native turn can encompass a user request, commentary, multiple tool invocations, and a final answer. A normalized turn is a message row. Preserve both native IDs and normalized indices rather than treating them as interchangeable.

2.2 Conversion and query pipeline

Native JSONL, immutable

|

```

Discover / locate / source identity
|
ConvertLocator -> ConvertRecords
|
parseSessionJSONL
|
Session {turns, tool_calls, metadata}
|
Archive publication + manifest + receipt
|
SQLite materialization / cache
|
SQL and embedded history commands
|
File timeline, failure counts, session recovery

```

pkg/adapters/codex/discover.go locates sources and extracts activity metadata. ConvertLocator reads native records; InspectSource establishes identity. ConvertRecords selects the parser and constructs the shared session. Archive publication is a separate command-layer responsibility. The query engine loads converted archives; it cannot recover native events the adapter discarded.

pkg/minitrace/schema.go defines the portable model. A tool has an ID, optional emitting-message index, input fields, output fields, and framework metadata. pkg/minitracedb/schema.go promotes common fields into SQL columns. arguments_json and raw_json provide escape hatches, but a copied opaque string is not the same as a properly normalized execution.

The important consequence is that this is primarily an adapter problem with downstream contract implications. Fixing SQL alone cannot restore absent turns or recover a reliable subprocess exit code from a malformed representation.

3. Concrete observed failure

3.1 Outer wrapper: retained but not interpreted

The native main session is 01a07757-8895-76d2-bcc5-fd9ebea379d5. At native line 1733 in the inspected source, call call_9uRzjH3Ug1zKzhJuUfBlrcnY uses custom_tool_call, name exec, with JavaScript source in input. The following is abbreviated; the test body and a second printing command are omitted:

```

text(await tools.exec_command({
  cmd: "cat > workbench/tests/test_embedding.py ...\\n"
    + "workbench/.venv/bin/pytest workbench/tests -q"
}));

```

The corresponding SQL row has:

```

{
  "tool_name": "exec",
  "operation_type": "OTHER",
  "command": null,
  "file_path": null,
  "exit_code": null,
  "success": 1
}

```

The source string survives in `arguments_json.input`. The row describes an outer orchestration call, not each executed shell command inside it. Searching that string can shortlist evidence, but cannot distinguish conditional code that never ran, independently failing calls, or output association across concurrent executions.

3.2 Structured execution: already available upstream

Native line 1735 contains a separate completed `CommandExecution` item with ID `exec-34f6356f-2e90-4b81-a3bf-0eda0c1c0556`. It records the shell `argv`, `cwd`, `exit_code: 0`, and `stdout 2` passed in `0.05s`. Its shell command creates the test file, appends to `.gitignore`, then runs `pytest`.

This structured event is stronger evidence than parsing the JavaScript wrapper. It proves execution and supplies an outcome. Prefer this record as the authoritative subprocess representation. Do not execute JavaScript from a transcript to discover its behavior.

Even a zero shell exit code does not prove every preceding operation succeeded: shells can run multiple statements without fail-fast behavior. Expose the shell outcome and retain the complete command; do not promote it into verified per-statement success.

3.3 Missing messages and malformed output

The current parser handles old `user_message` and `agent_message` event variants. It does not handle `response_item/message` or `event_msg/item_completed` in the persisted-session branch. This explains why the native messages disappear and pending calls fall back to emitting index zero even when no normalized message exists.

There is a separate output bug. `custom_tool_call_output.output` can be an array of content blocks. The parser currently passes it through `stringValue`; non-string values use `fmt.Sprintf`. This creates strings resembling `map[text: ... type:input_text]`, not JSON. Structured output boundaries and nested exit metadata are lost before `parseFunctionOutput` receives the value.

Finally, `buildCodexResponseToolCall` initializes `success` to `true`. If output parsing finds no exit code, that value can remain `true`. Thus the earlier explanation that success means outer completion was too strong: in this implementation it may merely be a default. Unknown outcome must not be reported as verified success.

4. Code navigation and API reference

Paths below are relative to the `go-minitrace` repository. Line numbers are inspection anchors, not a substitute for symbol search after edits.

- `pkg/adapters/codex/convert.go:58`, `ConvertLocator`: adapter entry point from a locator; reads and converts one native source.
- `pkg/adapters/codex/convert.go:83`, `InspectSource`: source identity and provenance inspection. Do not regress parent/child identity precedence.
- `pkg/adapters/codex/convert.go:171`, `ConvertRecords`: in-memory conversion entry point suitable for fixtures and unit tests.
- `pkg/adapters/codex/convert.go:398`, `parseSessionJSONL`: current event dispatch, pending calls, usage, and turn association. Primary repair site.
- `pkg/adapters/codex/convert.go:717`, `parseExecJSONL`: separate exec-stream parser. Examine for reusable semantics, but do not assume its event shape is identical to persisted `item_completed`.
- `pkg/adapters/codex/convert.go:953`, `buildCodexResponseToolCall`: preserves custom input and constructs outer call metadata.

- pkg/adapters/codex/convert.go:1003, applyCodexFunctionOutput: output truncation and exit/success handling. Change its input contract to accept structured data, not just a pre-flattened string.
- pkg/adapters/codex/convert.go:1029, commandForFunction: handles exec_command, write_stdin, and apply_patch; does not interpret exec JavaScript.
- pkg/adapters/codex/convert.go:1042, filePathForFunction: extracts selected direct tool paths; one scalar cannot represent every file in a shell script or patch.
- pkg/adapters/codex/convert.go:1165, classifyOperationFromCommand, and :1196, classifyFunction: current operation classification boundaries.
- pkg/adapters/codex/convert.go:1260, parseFunctionOutput: current string-output parsing; retain existing covered inputs while adding typed block support.
- pkg/adapters/codex/convert_test.go: existing session, exec, identity, metadata, and modern-tool tests. Extend rather than replacing coverage.
- pkg/minitrace/schema.go:140: ToolCall, ToolCallInput, and ToolCallOutput; Success is currently a non-nullable Go bool.
- pkg/minitracedb/schema.go:313: normalized tool_calls; SQL success is nullable even though the archive bool is not. Also inspect filesTable and turnToolCallsTable.
- pkg/minitracedb/materialize.go and convert.go: archive-to-row mapping; verify additional execution and file evidence reaches SQL.
- pkg/minitracecmd/core/history/file-history.js: consumer of structural file evidence. Test multi-target extraction and shell classification with actual results.
- skills/go-minitrace-transcript-analysis/SKILL.md: analyst workflow and caveats to update after the fix is demonstrated.

CLI interfaces for verification are `convert codex --source-session`, `query run --archive-glob --sql-file`, `validate --archive`, and `history file-history`. Inspect installed command help before using flags. Prefer `go run ./cmd/go-minitrace` during implementation to ensure the tested binary matches edited source.

5. Proposed architecture

5.1 Normalize events before building the portable session

Introduce an internal event-normalization stage with typed records for messages, outer tool invocations, executions, and outputs. This is an internal implementation proposal, not an additional public backend or compatibility layer. Existing supported formats remain supported through their existing dispatch.

```
response message -----+--> message identity reconciliation --> turns
item message -----+
```

```
custom exec call -----> orchestration record
                        |
                        explicit association only
                        |
CommandExecution -----> executed command record --> tool_calls
                        |
                        structural file evidence --> files
```

```
output content blocks ----> typed outcome + readable text
```

Keep native item ID, native turn ID, source record ordinal/line, event type, and timestamps as provenance. Use framework metadata for Codex-specific details until a shared schema requirement is justified. Do not blindly copy base instructions, encrypted reasoning, or image payloads into new metadata.

5.2 Message deduplication and linkage

Prefer `response_item/message` as the canonical representation when present; use `item-completed` messages as fallback. Reconcile by native message ID when available. Some user-message copies have different IDs; only merge them using explicit structural evidence or a narrowly defined same-turn, adjacent-event duplicate rule. Identical text alone is insufficient: a user can legitimately say “continue” twice.

```
for record in source order:
    decode supported message representation
    retain native message ID and native turn ID
    register candidate with source ordinal
```

```
reconcile equivalent message representations
assign stable normalized message indices
link calls to their actual emitting message if established
otherwise leave emitting index null and retain native turn ID
```

Do not attach all calls to a later final answer just because it is the next assistant message. When the source lacks a tool-emitting message, preserve that uncertainty. Repeated conversion of identical input must generate identical identities and associations.

5.3 Executions versus orchestration

Preserve the outer exec invocation as orchestration evidence and create executed-operation records from structured `CommandExecution` items. A stable derived ID can include the native item ID, namespaced to avoid collision with response call IDs. Deduplicate repeated lifecycle notifications by execution identity, not by identical command text or process ID alone.

Preserve `argv` and `cwd` separately. A shell `argv` ending in a script has a natural normalized command: the script argument. Do not flatten arbitrary `argv` with spaces and lose quoting. For non-shell commands retain `argv` in metadata and render a display string only with an explicitly documented quoting policy.

When no explicit parent-call linkage exists, keep a native-turn relationship rather than guessing which wrapper owned an execution based on timestamps. Parallel JavaScript promises invalidate positional output matching.

Counting needs a contract: outer calls count as orchestration, child executions as execution activity, and file touches as a separate measure. If both live in `tool_calls`, annotate record kind and update analytics so totals do not claim both are independent model-issued tools. Any proposed shared operation type or schema change requires explicit review; do not silently change the meaning of existing metrics.

5.4 Output and outcome semantics

Decode output values before converting them to text:

```
if output is a string:
    apply established string-result parser
else if output is content blocks:
    preserve block kinds and readable text boundaries
    inspect text blocks for recognized structured result envelopes
    retain independent subprocess outcomes independently
else:
    preserve bounded structured data and report unsupported shape
```

apply authoritative execution exit code if present
truncate display text only after extracting metadata

A JSON-looking arbitrary stdout line is not automatically a transport envelope. Parse only known envelope contexts and validate keys/types. Images should become attachment signals, not base64 embedded in SQL cells. Preserve truncation metadata and a source reference so analysts know when output is incomplete.

Represent pending, completed-success, completed-failure, cancelled, and unknown distinctly. The Go bool currently prevents a faithful unknown value. Decide explicitly between a reviewed nullable success field and a separately modeled outcome status with query changes. Do not default unknown to false either: that inflates failure counts. This cross-schema decision is a gate before implementation, not an invitation to add an unsolicited compatibility shim.

5.5 File operations without invented writes

One subprocess can read many files, write two files, and run tests. A scalar `file_path` is a convenience, not a complete ledger. Preserve multiple structured targets using the existing file/event projection where possible; review its schema and consumers before adding fields.

Recognize patch Add/Update/Delete headers and supported literal shell redirections. Keep source paths and resolve relative paths against the execution cwd, decoding a `file:// cwd` correctly. Do not use the session's initial cwd when a command has an explicit different cwd.

Shell extraction is evidence of an attempted operation. A search containing a filename is not a read. A heredoc may contain quoted shell commands that are data, not executed statements. Use a proper shell syntax parser or a deliberately narrow, tested grammar; avoid a regex that classifies every `>` inside arbitrary source code as a write. Dynamic paths stay unresolved with a diagnostic.

6. Safe treatment of wrapper-only sources

Structured execution events should be the first solution. Where only JavaScript source exists, preserve the wrapper and warn that semantic operations are unavailable. Optional static analysis can recognize literal calls and label them as intended operations, but must not equate them with execution.

Never evaluate transcript JavaScript using Goja, Node, or a shell. This repository has a JavaScript query runtime, but its existence does not authorize running historical tool input. A transcript can contain destructive commands, network calls, or untrusted injected content. Static parsing must be bounded by size and nesting limits and have no filesystem or network side effects.

A conditional false branch containing a write is the crucial negative test. If the analyzer reports that file as written, the design is unsafe for attribution even if its positive demos look convincing.

7. Implementation sequence and review gates

Phase A: Freeze reproducible evidence

Create small synthetic JSONL fixtures modeling the observed shapes. Include legacy supported events, paginated messages, an outer wrapper, a successful execution, a failing execution, and typed output blocks. Keep private native transcripts outside repository fixtures. Record the source commit and CLI version used for before/after measurements.

Run conversion into a new directory; never modify native logs. Save SQL reporting turns, operations, paths, outcomes, and orphan links. A fixture with a user and assistant message must not silently produce zero turns without a fidelity warning.

Phase B: Restore messages

Implement typed message extraction and duplicate reconciliation. Add tests for distinct repeated messages, representation duplicates, missing IDs, multi-block text, and stable order. Ensure all non-null emitting indices exist. Inspect native turn association separately from normalized message association.

Phase C: Normalize executions and output

Add structured execution recognition, lifecycle reconciliation, and output-block decoding. Resolve the outcome/schema design first. Test output-before-call, repeated completion, missing output, cancellation, shell exit failure, and conflicting metadata. Preserve conflicting evidence with a warning instead of silently overwriting it.

Phase D: File and metrics integration

Project multiple attempted touches, preserve cwd and evidence kind, and update history/failure queries. Measure orchestration versus execution counts separately. Reconvert an identical fixture twice and verify stable IDs; import both and ensure the normal archive collision rules apply rather than producing duplicate rows.

Phase E: Documentation and real-source acceptance

Run package tests and an end-to-end CLI test, then reconvert the private main source to a fresh investigation directory. Compare messages and structured executions against an independent native event inventory. Recheck source identity, receipts, and query cache invalidation. Update skill caveats only for demonstrated fixes; unsupported shapes should remain documented.

Suggested commands:

```
go test ./pkg/adapters/codex ./pkg/minitracedb ./pkg/minitracecmd
go test ./...
go run ./cmd/go-minitrace convert codex \
  --source-session /tmp/codex-fixture.jsonl \
  --output-dir /tmp/codex-fidelity-after
go run ./cmd/go-minitrace query run \
  --archive-glob '/tmp/codex-fidelity-after/active/*/*.minitrace.json' \
  --sql-file /tmp/codex-checks.sql --output json
```

8. Acceptance tests and invariants

- A paginated message fixture produces its expected user and assistant text once, with stable ordering.
- Repeated genuine user text remains repeated; mirrored representations do not double counts.
- A completed command preserves argv, cwd, stdout, exit code, and native execution ID.
- A failed child stays failed even when its wrapper completes successfully.
- Missing completion remains unknown and does not become a passing tool result.

- A shell command writing two files exposes two attempted touches, without inventing per-statement success.
- A multi-file patch exposes all structural targets; prose and grep-only mentions do not become writes.
- Array-valued outputs never become Go map[...] display strings.
- Truncation occurs after outcome parsing and carries a recoverable provenance reference.
- Every non-null tool-to-turn reference names an existing normalized turn.
- Duplicate lifecycle events yield one execution, while repeated real executions remain distinct.
- Parent and guardian native identities remain separate. Guardian quoted histories never generate execution rows.
- Unsupported event shapes yield explicit fidelity diagnostics instead of a deceptively clean archive.

SQL assertions should include orphan checks:

```
SELECT tc.tool_call_id, tc.emitting_turn_index
FROM tool_calls tc
LEFT JOIN turns t
  ON t.session_id = tc.session_id
  AND t.turn_index = tc.emitting_turn_index
WHERE tc.emitting_turn_index IS NOT NULL
      AND t.turn_index IS NULL;
```

Expected result: no rows. Also group by tool name and operation type, and inspect the actual example execution rather than judging success from aggregate counters alone.

9. Risks, unresolved decisions, and handoff

The largest design risks are double-counting mirrored records, inventing wrapper-child associations, overstating shell success, and changing public success semantics without updating consumers. Resolve those with explicit contracts before optimizing throughput. Native format labels are useful hints but not proof that all records conform to one schema.

The repository already has identity/collision handling and normalized SQLite infrastructure. This ticket should extend evidence fidelity within that architecture, not reintroduce DuckDB or add a second query engine. New fields require a review of archive schema, materialization, cache invalidation, presets, and UI consumers.

A prior archive validator run reported source-unavailable for a literal tilde-prefixed provenance path although the native file exists. That is a separate path-expansion issue; record it as a follow-up rather than claiming it proves missing source data or blocking this adapter design.

For a new intern, start with parseSessionJSONL and the synthetic fixture, then read the shared tool schema and SQL tables. Make one failing assertion at a time: restore the message, preserve the execution, establish its outcome, and finally expose its file evidence. The finished system should let an analyst answer where a command ran and what it reported without interpreting JavaScript or manually reading thousands of native lines.